

4.1 Introduction to Streams Concepts

The stream is the sequence of data elements which flows in a group. The big data analytics process the data which is stored in databases or generated at real time. Traditionally, the data used to be processed in batches which was stored in databases. The Batch processing is nothing but processing of block of data that have been stored in a database over a period of time. Such data contains millions of records generated in a day which are stored as a file or record and undergo processing at the end of the day for various kinds of analysis. It is capable to processes huge volumes of stored data with longer periods of latency. For example, processing all the transactions performed by a financial firm in a week for getting analytics.

The stream is communication of bytes or characters over the socket in a computer network. The stream processing is another big data technology which is used to query continuous data stream generated in a real-time for finding the insights or detect conditions and quickly take actions within a small period of time. The data streaming is the process of sending continuous data rather than in batches. In some applications, the data comes as a continuous stream of events. If we use batch processing in such applications, then we need to stop the data collection somewhere and need to store data in a batch for processing in that case if next batch comes for process then we have to aggregate the results of previously processed batches which is quite difficult and time consuming. In contrast, the stream processing supports never ending data streams that can detect patterns and inspect multiple levels of results on real time data without being stored in batches with simpler aggregation of processed data. For example, with stream processing, you can receive an alert when the stock market prices cross to the threshold or get the notification when temperature reaches to the freezing point by querying data streams coming from a temperature sensor. The data streaming is ideally used for time series analysis or detecting hidden patterns over the time and it can simultaneously process the multiple streams. In the data stream model, individual data items may be relational tuples, for example, network measurements, call records, web page visits, sensor readings and so on. However, their continuous arrival in multiple, rapid, time-varying, possibly unpredictable and unbounded streams appear to yield some fundamentally new research problems.

4.1.1 Applications of Data Streaming

The popular applications of data streaming are :

- a) In E-commerce site, to find the anomalous behavior in the data stream they stream

the clickstream records and generates a security alert if the clickstream shows suspicious behavior.

- b) In financial institutions, to tracks the market changes the customer portfolios are adjusted based on configured constraints.
- c) In power power grid, the alert or notification is generated based on throughput when certain thresholds are reached.
- d) In news source, the articles that are relevant to the audience are generated by analyzing the clickstream records from various sources based on their demographic information.
- e) In network management and web traffic engineering, the streams of packets are collected and processed to detect anomalies.

4.1.2 Sources of Streamed Data

There are various sources of streamed data which provides data for stream processing.

The sources of streaming data ranges from computer applications to the Internet of things (IOT) sensors. It include satellites data, sensors data, IOT applications, websites, social media data etc. The various examples of sources of stream data are listed below

- a) **Sensor data** : Where data receives from different kinds of wired or wireless sensors. For example, the real time data generated by temperature sensor provided to the stream processing engine for taking action when threshold meets.
- b) **Satellite image data** : Where data receives from satellites to the to earth streams which consist of many terabytes of images per day. The surveillance cameras are fitted in a satellite which produces images for processing streamed at station on earth.
- c) **Web data** : Where the real time streams of IP packets generated on internet are provided to the switching node which runs queries to detect denial-of-service attacks or other attacks are then reroute the packets based on information about congestion in the network.
- d) **Data in an online retail stores** : Where retail firm data collect, store and process data about product purchase and services by particular customer to understand the customers behavior analysis
- e) **Social web data** : Where data generated through social media websites like Twitter. Facebook is used by third party organization for sentimental analysis and prediction of human behavior.

The applications which use data streams are,

- Realtime maps which use location-based services to find nearest point of interest
- Location based advertisement or notifications
- Watching streamed videos or listening streamed music
- Subscribing online news alerts or weather forecasting service
- Monitoring
- Performing fraud detection on live online transaction
- Detection of anomaly on network applications
- Monitoring and detection of potential failure of system using network monitoring tools
- Monitoring embedded systems and industry machinery in real-time using surveillance cameras
- Subscribing real-time updates on social medias like twitter, Facebook etc.

4.2 Stream Data Model and Architecture

The stream data model is responsible for receiving and processing the real-time data over analytical platforms. The stream data model uses data stream management system unlike database management systems. It consists stream processors for managing and processing the data streams. The input for stream processor is provided by the applications which allows multiple streams to be enter into the system for processing.

4.2.1 Data Stream Management System

The traditional relational databases are intended for storing and retrieving records of data that are static in nature. Further these databases do not perceive a notion of time unless time is added as an attribute to the database during designing the schema itself. While this model was adequate for most of the legacy applications and older repositories of information, many current and emerging applications require support for online analysis of rapidly arriving and changing data streams. This has prompted to build new models to manage streaming data. This has resulted in data stream management systems (DSMS), with an emphasis on continuous query languages and query evaluation. Each input stream in data stream management system poses different data types and data rates. The typical architecture of data stream management system is shown in Fig. 4.2.1.

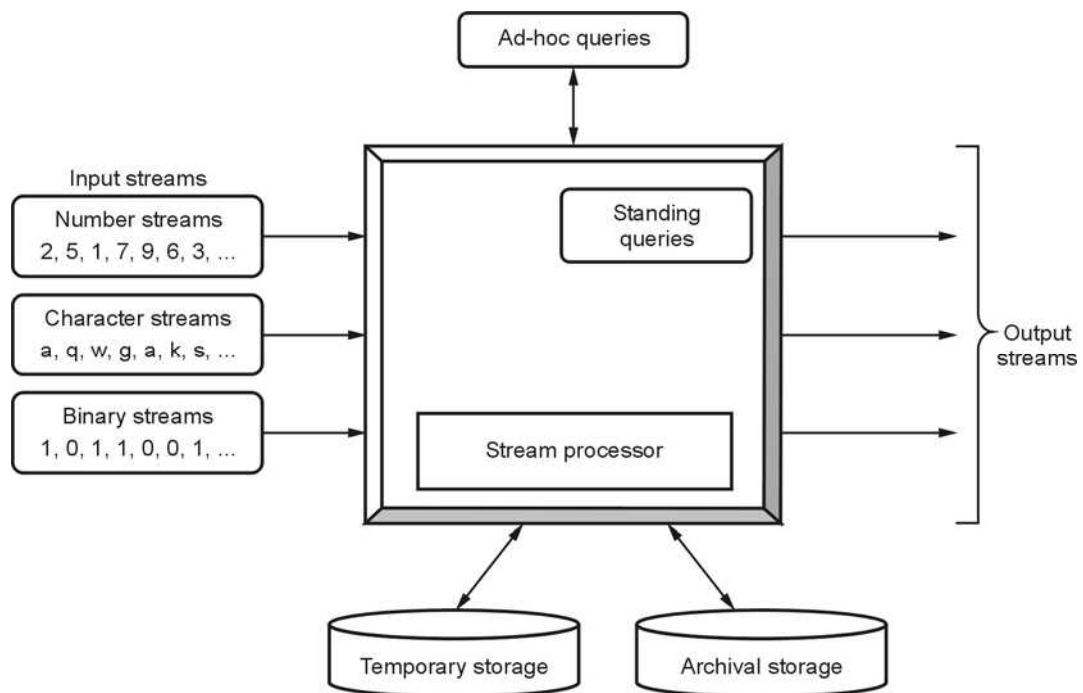


Fig. 4.2.1: Architecture of data stream management system

The streams which are inputted to the stream processor has to be stored in a temporary store or working store. The temporary store in data stream model is a transient store used for storing the parts of streams which can be queried for processing. The temporary store can be a disk, or main memory, depending on how fast queries to be processed. The results of the queries are stored in a large archival storage but archival data cannot be used for query processing but can be used in special circumstances. A streaming query is a continuous query that executes over the streaming data. They are similar to database queries used for analyzing data and differ by operating continuously on data as they arrive incrementally in real-time. The stream processor supports two types of queries namely ad-hoc queries and standing queries. In ad-hoc query, the variable dependent results are generated where each query generates different results depending on the value of the variable. The ad-hoc query uses common approach to store a sliding window of each stream in the working or temporary store. It doesn't allow to store all the streams entirely as it expects to answer arbitrary queries about the streams by storing appropriate parts or summaries of streams. They are intended for a specific purpose in contrast to a predefined query.

Alternatively, the standing queries are continuous query that executes over the streaming data whose functions are predetermined. For standing query, each time a new stream is arrives and produces the aggregate results.

4.2.2 Data Streaming Architecture

A streaming data architecture is a framework for processing huge volumes of streaming data from multiple sources. As traditional data solutions concentrated on consuming and processing data in batches while streaming data architecture consumes data immediately as it is produced, store it in storage mediums and perform real-time data processing, manipulation and analytics. Most of the streaming architectures are built on solutions, specific to the problems such as stream processing, data integration, data storage and real-time analytics. The generalized streaming architecture composed of four components like Message Broker or Stream Processor, ETL tools, Query Engine and streaming data storage shown in Fig. 4.2.2.

The first component of data streaming architecture is Message Broker or Stream Processors. They are the producer of the data which translate the streams into a standard message format. The other components in the architecture can consume the messages passed by the broker. The popular legacy message brokers are RabbitMQ and Apache ActiveMQ which are based on Message Oriented Middleware while the latest messaging platforms for stream processing are Apache Kafka and Amazon Kinesis.

The second component of the data stream architecture is batch or real-time ETL (Extract, Transform and Load) tools that streams data from one or more message brokers and aggregate or transform them into a well-defined structured before data can be analyzed with SQL-based analytics tools.

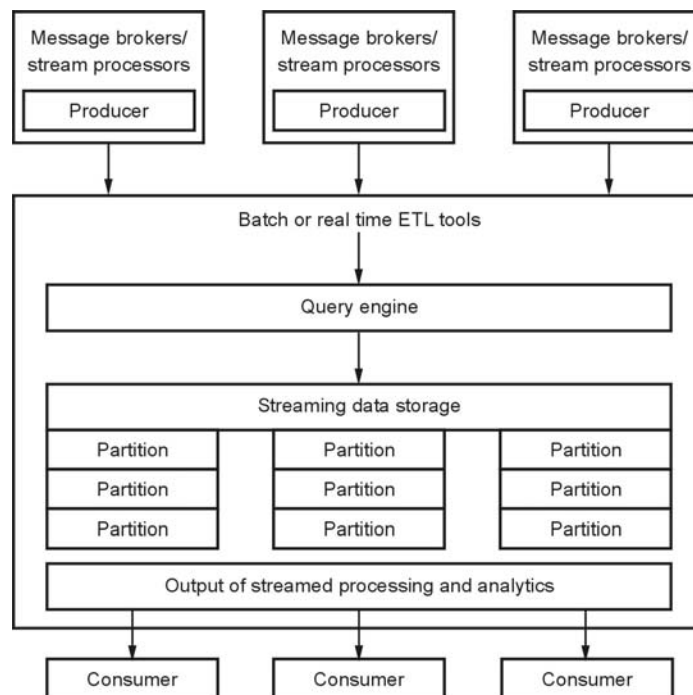


Fig. 4.2.2 : Generalized data streaming architecture

The ETL platforms receives queries from users, based on that it fetches the events from message queues and applies the query stream data to generate a result by performing additional joins, transformations on aggregations. The result may be an API call, an action, a visualization, an alert, or in some cases a new data stream. The popular ETL tools for streaming data are Apache Storm, Spark and Flink and Samza.

The third component of the data stream architecture is Query Engine which is used once streaming data is prepared for consumption by the stream processor. Such data must be analyzed to provide valuable insights, while the fourth component is streaming data storage, which is used to store the streaming event data into different data storage mediums like data lakes.

The stream data processing provides several benefits like able to deal with never-ending streams of events, real-time data processing, detecting patterns in time-series data and easy data scalability while some of the limitations of stream data processing are network latency, limited throughput, slow processing. Supporting on window sized streams and limitations related to In-memory access to stream data.

The common examples of data stream applications are

- **Sensor networks** : Which is a huge source of data occurring in streams and are used in numerous situations that require constant monitoring of several variables, based on which important decisions are made.
- **Network traffic analysis** : In which, network service providers can constantly get information about Internet traffic, heavily used routes, etc. to identify and predict potential congestions or identify potentially fraudulent activities.
- **Financial applications** : In which the online analysis of stock prices is performed which is used for making the sell decisions about the product, quickly identifying correlations with other products, understand fast changing trends and to an extent forecasting future valuations about the product.

The Queries over continuous data streams have much in common with queries in a traditional DBMS. There are two types of queries can be identified as typical over data streams namely One-time queries and Continuous queries :

- a) **One-time queries** : One-time queries are queries that are evaluated once over a point-in-time snapshot of the data set, with the answer returned to the user. For example, a stock price checker may alert the user when a stock price crosses a particular price point.
- b) **Continuous queries** : Continuous queries, on the other hand, are evaluated continuously as data streams continue to arrive. The answer to a continuous query is

produced over time, always reflecting the stream data seen so far. Continuous query answers may be stored and updated as new data arrives, or they may be produced as data streams themselves.

4.2.3 Issues in Data Stream Query Processing

Apart from benefits, there are some issues in data stream query processing which are explained as follows

- a) **Unbounded memory requirements** : Since data streams are potentially unbounded in size, the amount of storage required to compute an exact answer to a data stream query may also grow without bound. Algorithms that use external memory are not well-suited to data stream applications since they do not support continuous queries. For this reason, we are interested in algorithms that are able to confine themselves to main memory without accessing disk.
- b) **Approximate query answering** : When we are limited to a bounded amount of memory, it is not always possible to produce exact answers for the data stream queries; however, high-quality approximate answers are often acceptable in lieu of exact answers.
- c) **Sliding windows** : One technique for approximate query answering is to evaluate the query not over the entire past history of the data streams, but rather only over sliding windows of the recent data from the streams. Imposing sliding windows on data streams is a natural method for approximation that has several attractive properties but for many applications, sliding windows can be a requirement needed as a part of the desired query semantics explicitly expressed as a part of the user's query.
- d) **Blocking operators** : A blocking query operator is a query operator that is unable to produce an answer until it has seen its entire input.

4.3 Stream Computing

The stream computing is a computing paradigm that reads data from collections of sensors in a stream form and as a result it computes continuous real-time data streams. The stream computing enables graphics processors (GPUs) to work in coordination with low-latency and high-performance CPUs to solve complex computational problems. The data stream in stream computing has sequence of data sets and a continuous stream carries infinite sequence of data sets. Stream computing can be applied on high velocity stream of data from real time sources such as market data, mobile, sensors, click Stream and even transactions. It empowers organizations to analyze and follow up on rapidly changing data in real time, upgrade existing models with new bits of insights, capture

analyze and act on insights, and to move from batch processing to real time analytical decisions. The stream computing supports low-latency velocities and massively parallel processing architectures to obtain the useful knowledge from big data. Consequently, the stream computing model is a new trend for high-throughput computing in the big data analytics. The different organizations who uses stream computing are telecommunication, health care, utility companies, municipal transits, security agencies and many more. The two popular use cases of stream computing are distribution load forecasting, conditional maintenance and smart meter analytics in energy industry and monitoring a continuous stream of data and generate alerts when intrusion is detected on a network through a sensor input.

4.3.1 Stream Computing Architecture

The architecture of stream computing consists of five components namely: Server, integrated development environment, database connectors, streaming analytics engine and data mart. The generalized architecture of stream computing is shown in Fig. 4.3.1.

In this architecture, the server is responsible for processing the real-time streaming event data with high throughputs and low latency. The low latency is provided by means of processing the streams in a main memory.

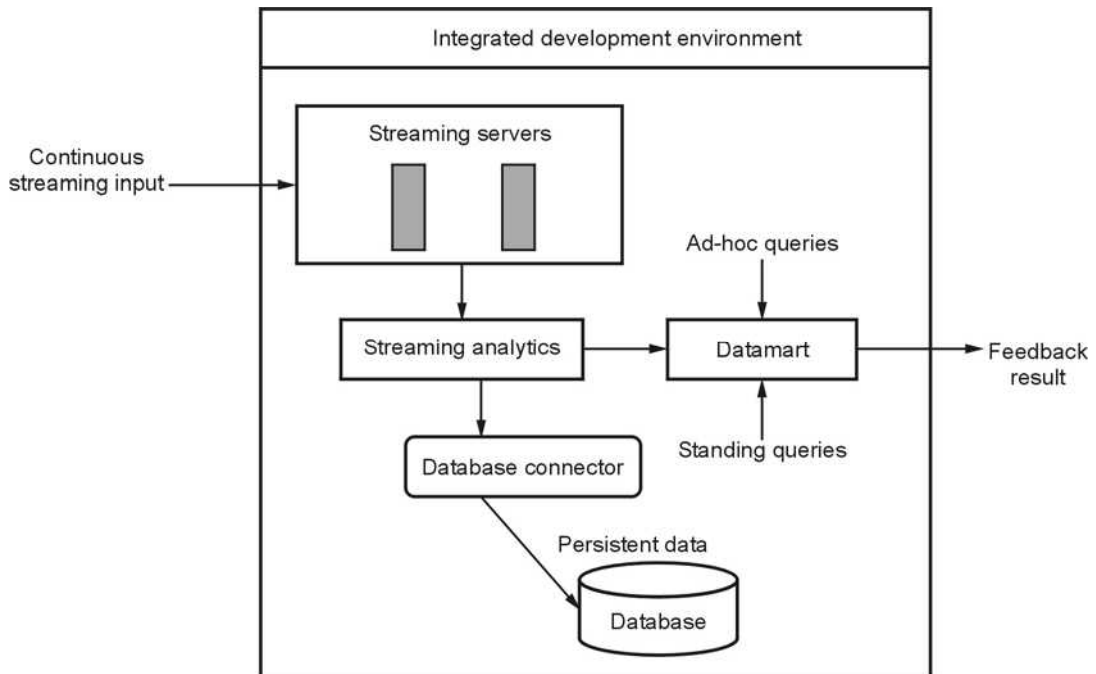


Fig. 4.3.1 : Generalized architecture of stream computing

The Integrated development environment (IDE) is used for debugging and testing of stream processing applications that processes streams using streaming operators, visual development of applications, provides filtering, aggregation, correlation methods for streamed data along with user interface for time windows analysis. The database connectors are used for providing rule engines and stream processing engines for processing a streamed data with multiple DBMS features. Common main memory DBMS and rule engines are to be redesigned to use in stream computing. The streaming analytics engine allows management, monitoring, and real-time analytics for real-time streaming data and data mart is used for storing live data for processing with additional feature like operational business intelligence. It also provides automated alerts for the events.

4.3.2 Advantages of stream computing

The advantages of stream computing as listed as follows

- It provides simple and extremely complex analytics with agility
- It is scalable as per computational intensity
- It supports a wide range of relational and non-relational data types
- It can analyze continuous, massive volumes of data at rates up to petabytes
- Performs complex analytics of heterogeneous data types including text, images, audio, voice, VoIP, video, web traffic, email, GPS data, financial transaction data, satellite data, sensors, and any other type of digital information that is relevant to your business.
- Leverages sub-millisecond latencies to react to events and trends as they are unfolding, while it is still possible to improve business outcomes.
- It can seamlessly deploy applications on any size computer cluster and adapts to work in rapidly changing environment.

4.3.3 Limitations of Stream Computing

- In an extreme case security and data confidentiality is the main concern in Stream computing.
- The flexibility, resiliency and data type handling are the serious considerations in stream computing.

In data stream processing, the three important operations used are sampling, filtering and counting distinct elements from the stream which are explained in next subsequent sections.

4.4 Sampling Data in a Stream

The sampling in a data stream is the process of collecting and representing the sample of the elements of a data stream. The samples are usually much smaller element of entire stream, but designed to retain the original characteristics of the stream. Therefore, the elements that are not stored within the sample are lost forever, and cannot be retrieved. The sampling process is intended for extracting reliable samples from a stream. The data stream sampling uses many stream algorithms and techniques to extract the sample, but most popular technique is hashing.

It is used when we have multiple subsets of a stream and want to run the query on that which can retrieve statistically representative of the stream as a whole. In such cases ad-hoc queries can be used on the sample along with hashing.

For example, suppose, we want to study a user's behavior on search engine and search engine receives multiple stream of queries. Here, we assume that the stream consists of tuples user, query and time. So, if we run an ad-hoc query to find out what portion of the typical user's search queries were repeated over the past month?" In that case, approach would be to generate a random number between 0 to 9, in response to each search query. Here assume that the tuple will be stored if and only if the random number is 0, so as to store the average, 1/10th of queries of each user. But due to the statistical fluctuations into the data noise gets introduced when users issue large numbers of queries. However, this scheme gives wrong answer to the query asking for the average number of duplicate queries for a user. So, we consider the S search queries one time in a month and T search queries twice.

Therefore for 1/10th sample of queries, user can expect S/10 of the search and for twice search queries, it would be $T/10 \times 1/10 = T/100$ to fraction T times the probability that both occurrences of the query will be in the 1/10th sample. For, full stream the query about the fraction of repeated searches would be $T/(S+T)$.

To find representative search we use In and Out keywords. So, if we see the previous search records for the user during the current search then we do not do anything but if we have no search record for the user, then we generate a random integer between 0 and 9. If the number generated is 0 then we add this user to our list with value "in," otherwise we add the user with the value "out."

This method works well as long as we keep the list of all users and in/out decision in main memory. By using a hash function, one can avoid keeping the list of users such that for each user name hash to one of ten buckets, 0 to 9. Therefore, if the user hashes to bucket 0, then accept this search query for the sample, and if not, then not. Effectively, we use the hash function as a random number generator and without storing the in/out decision for any user, we can reconstruct that decision any time a search query by that user arrives.

The generalized sampling problem consists of tuples with n components for the streams. A subset of the components are the key components, on which the selection of the sample will be based. In our example, the user, query, and time are the subsets and users are in the key. However, we can use sample of queries on key attributes to get the outcome.

In general, to generate a samples of size a/b where a is the key and b are the tuples, we hash the key value a for each tuple to b buckets, and accept the tuple for the sample if the hash value is less than a . The result will be a sample consisting of all tuples with certain key values and the selected key values will be approximately a/b of all the key values appearing in the stream. While sampling methods reduce the amount of data to process, and, by consequence, the computational costs, they can also be a source of errors. The main problem is to obtain a representative sample, a subset of data that has approximately the same properties of the original data.

4.4.1 Types of Sampling

There are basic three types of sampling explained as follows

4.4.1.1 Reservoir Sampling

In reservoir sampling, the randomized algorithms are used for randomly choosing the samples from a list of items, where list of items is either a very large or unknown number. For example, imagine you are given a really large stream of data and your goal is to efficiently return a random sample of 1000 elements evenly distributed from the original stream. A simple way is to generate random integers between 0 and $(N - 1)$, then retrieving the elements at those indices will give the answer.

4.4.1.2 Biased Reservoir Sampling

In biased reservoir sampling is a bias function to regulate the sampling from the stream. In many cases, the stream data may evolve over time, and the corresponding data mining or query results may also change over time. Thus, the results of a query over a

more recent window may be quite different from the results of a query over a more distant window. Similarly, the entire history of the data stream may not be relevant for use in a repetitive data mining application such as classification. The simple reservoir sampling algorithm can be adapted to a sample from a moving window over data streams. This is useful in many data stream applications where a small amount of recent history is more relevant than the entire previous stream. This will give a higher probability of selecting data points from recent parts of the stream as compared to distant past. The bias function in sampling is quite effective since it regulates the sampling in a smooth way so that the queries over recent horizons are more accurately resolved.

4.4.1.3 Concise Sampling

Many a time, the size of the reservoir is sometimes restricted by the available main memory. It is desirable to increase the sample size within the available main memory restrictions. For this purpose, the technique of concise sampling is quite effective. Concise sampling exploits the fact that the number of distinct values of an attribute is often significantly smaller than the size of the data stream. In many applications, sampling is performed based on a single attribute in multi-dimensional data that type of sampling is called concise sampling. For example, customer data in an e-commerce site sampling may be done based on only customer ids. The number of distinct customer ids is definitely much smaller than “n” the size of the entire stream.

4.5 Filtering Streams

The data stream processing poses another approach called selection, or filtering. The filtering is the process of accepting the tuples in the stream that meets the selection criterion where accepted tuples are provided to another process as a stream and rejected tuples are dropped. In filtering, If the selection criterion is a based-on property of tuple then filtering would be easier but if selection criterion involves lookup for membership function in a set then it becomes hard to filter the stream and large to store in main memory.

The Hashes are the individual entries in a hash table that act like the index. The hash function is used to produce the hash values where input is an element containing complex data, and the output is a simple number that acts as an index to that element. A hash function is deterministic in nature because it produces the same number every time you feed it a specific data input.

Let us take an example, suppose we have a set {S} of one million allowed email addresses which are not to be spam. So, the stream consists of email address and the

email itself as a pair. As each email address consumes 20 bytes or more space, it is not reasonable to store the set S in main memory. Thus, we have to use disk to store and access that.

Suppose we want to use main memory as a bit array, then we need eight million bits array and have to run hash function h to eight million buckets from email addresses.

Since there are one million members of S , approximately $1/8$ th of the bits will be 1 and rest would be 0. Here, as soon as stream element arrives, we hash its email address, if hash value for stream element e-mail comes to 1 then we let the email through else we drop this stream element. But sometimes spam email will get through, so to eliminate every spam, we need to check for membership in set S those good and bad emails that get through the filter. The Bloom filter is used in such cases to eliminate the tuples which do not meet the selection criterion.

4.5.1 Bloom Filter

The purpose of the Bloom filter is to allow all the stream elements whose keys (K) lie in set (S) otherwise rejecting most of the stream elements whose keys (K) are not part of set (S). The basic algorithm of bloom filter consist of test and add methods where test is used to check whether a given element is in the set or not. If it returns the outcome as false then we conclude element is definitely not in the set, if it returns true then we consider the element is probably in the set and false positive rate is a function of the bloom filter' used to calculate size and the number of independent hash functions used. The add method simply adds an element to the set where removal is not possible without introducing false negative values, but extensions to the bloom filter are possible.

Typically, a Bloom filter algorithm has three basic steps given as follows :

- a) Select an array/vector of n bits, whose initial bits set to all 0's.
- b) The group of hash functions like $\{H_1, H_2, \dots, H_k\}$ where each hash function maps the "Key" (K) values to n buckets, corresponding to the n bits of the array.
- c) Make a set (S) of matched (m) key values.

In bloom filtering, at the first step we initialize the n bit array by setting all bits 0. Then starts with taking each key value (K) in set (S) and hash it using each of the m hash functions. The outcome is set to 1, if each bit is in $h_i(K)$ for some hash function h_i found then we conclude that some key value (K) are present in set (S).

To test a key (K) that arrives in the stream, check that all the hash functions $h_1(K)$, $h_2(K)$, $\dots$, $h_k(K)$ which has 1's in the bit-array. If all the values found to be 1's, then let

the stream element pass through else discard. That means, if one or more of these bits are remains 0, then K could not be found in S, so reject the stream element. So, to find out how many elements are passed we need to calculate the probability of a false positive outcomes, as a function of n bit-array length, m the number of members of set (S), and m number of hash functions.

Let us take an example, where we have a model which is used for throwing darts at the targets. Here, suppose we have T targets and D darts and there is a possibility of any dart is equally likely to hit any target. So, the analysis of how many targets can we expect to be hit at least once falls in one of the conditions given below :

- The probability of a given dart will not hit a given target would be $\frac{T-1}{T}$
- The probability of none of the D darts will hit a given target would be $\left(\frac{T-1}{T}\right)^D$
- With approximation, the probability that none of the y darts hit a given target would be $e^{-(D/T)}$.

4.6 Counting Distinct Elements in a Stream

After performing sampling and filtering on data stream, the third kind of processing is count-distinct problem. The sampling and filtering as used to calculate space needed per stream in a reasonable amount of main memory by using a variety of hashing and a randomized algorithm.

4.6.1 The Count-Distinct Problem

The count-distinct problem is used for finding the number of distinct elements in a data stream with repeated elements. Suppose stream elements are chosen from some universal set. We would like to know how many different elements have appeared in the stream, counting either from the beginning of the stream or from some known time in the past. A simple solution is to traverse the given array, consider every window in it and count distinct elements in the window.

For example : Given an array of size n and an integer k, return the of count of distinct numbers in all windows of size k. Where $k = 4$ and input array is {1, 2, 1, 3, 4, 2, 3}.

Here as window size $k = 4$, in the first pass the window would be {1, 2, 1, 3}. So, the count of distinct numbers in first pass is 3. In second pass, window would be {2, 1, 3, 4} and the count of distinct numbers in second pass is 4. In third pass, window would be {1, 3, 4, 2} and the count of distinct numbers in third pass is 4 and in fourth pass, window

would be {3, 4, 2, 3}, the count of distinct numbers in fourth pass is 3. Therefore, the final count of distinct numbers are 3, 4, 4, 3.

Let us take another example, suppose we want to find out how many unique users have accessed a particular website let's say Amazon in a given month based on gathering statistics. So, here universal set would be a set of logins and IP address which has sequences of four 8-bit bytes from which they send the query for that site. The easiest way to solve this problem is to keep the set in main memory which has list of all the elements in the stream and make them arranged in an search structure like hash table or search tree so as to add new elements quickly. But the problem here is to obtain an exact number of distinct elements appear in the stream. However, if the number of distinct elements is too large then we cannot store them in main memory. Therefore, the solution of this problem is to use several machines for handling only one or more number of the streams and store most of the data structure in secondary.

4.6.2 The Flajolet-Martin Algorithm

The Flajolet-Martin algorithm is used for calculating the number of distinct elements in a stream. It approximates the number of unique objects in a stream in single pass. It is possible to estimate the number of distinct elements by hashing the elements of the universal set to a bit-string. The basic property of a hash function is when it is applied to the same element, it generates same result.

The Flajolet-Martin algorithm is used in cases where large number of different elements lies in the stream with more different hash-values where one of these values will be "unusual." So, if the stream contains n elements and m of them unique, this algorithm runs in $O(n)$ time and needs $O(\log(m))$ memory space. The Flajolet-Martin algorithm is given as follows

Flajolet-Martin Algorithm :

- 1) Pick a hash function h that maps each of the n elements to at least $\log_2 n$ bits.
- 2) For each stream element x , let $r(x)$ be the number of trailing 0's in $h(x)$.
- 3) Record $R =$ the maximum $r(x)$ seen.
- 4) Estimate the count $= 2^R$

The steps for counting distinct elements in a stream using Flajolet-Martin algorithm is as follows :

Step 1 : Create a bit array/vector of length L and suppose there are n number of elements in the stream, such that $2L > n$.

Step 2 : The i^{th} bit in array/vector represents the hash function value whose binary representation ends in 0^i . So, initialize each bit to 0.

Step 3 : Generate a feasible random hash function that maps input string to natural numbers.

Step 4 : For each word in an input stream perform hashing and determine the number of trailing zeros, such that if the number of trailing zeros is k , set the k^{th} bit in the bit array/vector to 1.

Step 5 : Get the index of the first 0 (called R) in the bit array/vector when input is exhausted. In this way calculate the number of consecutive 1's. Here, we have seen 0, 00, ..., 0^{R-1} as the output of the hash function plus one.

Step 6 : Calculate the number of unique words as $2R/\varphi$, where φ is 0.77351.

Step 7 : This implies that our count can be off by a factor of 2 for 32 % of the observations, off by a factor of 4 for 5 % of the observations, off by a factor of 8 for 0.3 % of the observations and so on. As the standard deviation of R is a constant : $\sigma(R) = 1.12$. (as R can be off by 1 for $1 - 0.68 = 32\%$ of the observations, off by 2 for about $1 - 0.95 = 5\%$ of the observations and off by 3 for $1 - 0.997 = 0.3\%$ of the observations using the Empirical rule of statistics.)

Step 8 : To improve accuracy of this approximation algorithm, do the averaging that uses multiple hash functions and use the average R instead, Bucketing that uses multiple buckets of hash functions from the above step and use the median of the average R as averages are susceptible to large fluctuations and bucketing gives fairly good accuracy and use appropriate number of hash functions in the averaging and bucketing steps to get more accuracy. It means that the accuracy depends on number of hash functions where higher the accuracy more the hash functions and higher computation cost.

Example 1 : Given a stream $S = \{4, 2, 5, 9, 1, 6, 3, 7\}$ and hash function $h(x) = (ax + b) \bmod 32$. So, count the distinct elements in a stream using Flajolet-Martin (FM) algorithm and treat the result as a 5-bit binary integer.

In a given example, the hash function is given as $h(x) = (ax + b) \bmod 32$. So, to estimate the number of elements appearing in a stream, we have to use hash function to integer elements interpreted as binary numbers and find out 2 raised to the power of that which is the longest sequence of 0's seen in the hash value of any stream element is an estimate of the number of distinct elements.

Let us assume $a = 3$ and $b = 1$, therefore hash function $h(x)$ would be $3x+1 \bmod 32$.

So, calculate the hash value in binary format for each stream in S i.e. $S = \{4, 2, 5, 9, 1, 6, 3, 7\}$

$$h(4) = 3(4) + 7 \bmod 32 = 19 \bmod 32 = 19 = (10011)$$

$$h(2) = 3(2) + 7 \bmod 32 = 13 \bmod 32 = 13 = (01101)$$

$$h(5) = 3(5) + 7 \bmod 32 = 22 \bmod 32 = 22 = (10110)$$

$$h(9) = 3(9) + 7 \bmod 32 = 34 \bmod 32 = 2 = (00010)$$

$$h(1) = 3(1) + 7 \bmod 32 = 10 \bmod 32 = 10 = (01010)$$

$$h(6) = 3(6) + 7 \bmod 32 = 25 \bmod 32 = 25 = (11001)$$

$$h(3) = 3(3) + 7 \bmod 32 = 16 \bmod 32 = 16 = (10000)$$

$$h(7) = 3(7) + 7 \bmod 32 = 28 \bmod 32 = 28 = (11100)$$

Now let us find trailing number of 0's in each binary output by observing rightmost number of zeroes. So trailing zero's for given stream would be $\{0, 0, 1, 1, 1, 0, 4, 2\}$. Therefore, value of R would be maximum number of trailing zeros i.e. $R = 4$.

So, Number of distinct elements $(N) = 2^R = 2^4 = 16$.
--

Example 2 : Given a Stream $S = \{1, 3, 2, 1, 2, 3, 4, 3, 1, 2, 3, 1\}$ and hash function $h(x) = (6x+1) \bmod 5$, treat the result as a 5-bit binary integer.

So, calculate the hash value in binary format for each stream in $S = \{1, 3, 2, 1, 2, 3, 4, 3, 1, 2, 3, 1\}$

$$h(1) = (6 * (1)+1) \bmod 5 = 7 \bmod 5 = 2 = (00010)$$

$$h(3) = (6 * (3)+1) \bmod 5 = 19 \bmod 5 = 4 = (00100)$$

$$h(2) = (6 * (2) +1) \bmod 5 = 13 \bmod 5 = 3 = (00011)$$

$$h(1) = (6 * (1)+1) \bmod 5 = 7 \bmod 5 = 2 = (00010)$$

$$h(2) = (6 * (2) +1) \bmod 5 = 13 \bmod 5 = 3 = (00011)$$

$$h(3) = (6 * (3)+1) \bmod 5 = 19 \bmod 5 = 4 = (00100)$$

$$h(4) = (6 * (4)+1) \bmod 5 = 25 \bmod 5 = 0 = (00000)$$

$$h(3) = (6 * (3)+1) \bmod 5 = 19 \bmod 5 = 4 = (00100)$$

$$h(1) = (6 * (1)+1) \bmod 5 = 7 \bmod 5 = 2 = (00010)$$

$$h(2) = (6 * (2) +1) \bmod 5 = 13 \bmod 5 = 3 = (00011)$$

$$h(3) = (6 * (3)+1) \bmod 5 = 19 \bmod 5 = 4 = (00100)$$

$$h(1) = (6 * (1)+1) \bmod 5 = 7 \bmod 5 = 2 = (00010)$$

Now let us find trailing number of 0's in each binary output by observing rightmost number of zeroes. So Trailing zero's for given stream would be {1,2,0,1,0,2,5,2,1,0,2,1}. Therefore, value of R would be maximum number of trailing zeros i.e. R= 5.

So, Number of distinct elements (N) = $2^R = 2^5 = 32$.

Here, whenever we apply a hash function H to a stream element a, the bit string H(a) will end in some number of 0's. Assume this number as a tail length for a and H. Let N be the maximum tail length of the stream. Then we shall use estimate 2^N for the number of distinct elements seen in the stream. This estimate makes intuitive sense.

The Intuition here are

- a) The probability of a given stream element a has hash H(a) ending in at least n number of 0's is 2^{-n} .
- b) Suppose there are m distinct elements in the stream, then the probability that none of them has tail length at least n is $(1 - 2^{-n})^m$.

Here we conclude that If m is larger than 2^n , then the probability for finding tail length at least n approaches is 1 and If m is much less than 2^n , then the probability of finding a tail length at least n approaches is 0. But inappropriately there is a trap regarding the strategy for combining the estimates of m for the number of distinct elements obtained by using many different hash functions. As per our first Intuition, if we take the average of the values 2^N then get a value that approaches the true m hash function but the influence of overestimate has occurred on the average. Suppose value of 2^n is much larger than m, then some probability p to discover n to be the largest number of 0's at the end of the hash value for any of the stream elements m and the probability of finding n +1 for largest number of 0's would be at least p/2.

For the space requirement, as we know that during the reading on the stream, one integer per hash function needs to be kept in main memory. But as integer records the largest tail length for the hash function, it is difficult to find out the space requirement. Processing only one stream could use millions of hash functions which are far more than the estimates. So, the main memory constrain would be only the number of hash functions that are trying to process many streams at the same time.

4.7 Estimating Moments

The generalization of the problem of counting distinct elements in a stream is an interesting issue by itself. The problem, called computing "moments", involves the distribution of frequencies of different elements in the stream. The estimating moments involves the distribution of frequencies of various elements in the stream. Suppose a

stream consists of elements chosen from a universal set U which has ordered elements i and m_i be the number of occurrences of the i^{th} element. Then the k^{th} order moment of the stream is calculated as sum over all i.e.

$$F_k = \sum_{i \in A} (m_i)^k$$

Here, 0^{th} moment of the stream is sum of 1 for each $m_i > 0$; number of distinct elements. 1^{st} moment of the stream is the sum of all m_i , which must be the length of the stream. The 2^{nd} moment of the stream sum of the squares of the m_i^2 , which could be a surprise number (S) that measures the uneven the distribution of elements in the stream, m^2 describes the “skewness” of a distribution; smaller the value of M^2 , less skewed is the distribution.

For example, suppose we have a stream of length 100, in which eleven different elements are appeared. The most even distribution of these eleven elements would be 1 appearing 10 times and the 10 appearing 9 times each. In this case, the surprise number would be $1 \times 10^2 + 10 \times 9^2 = 910$. Here, we can't keep count for each element that appeared in a stream in main memory. So, we need to estimate the k^{th} moment of a stream by keeping a limited number of values in main memory and computing an estimate from these values.

Examples :

Consider the following data streams and calculate the surprise number :

1) 5,5,5,5,5 $\rightarrow$ Surprise number = $5 \times 5^2 = 125$

2) 9,9,5,1,1 $\rightarrow$ Surprise number = $(2 \times 9^2 + 1 \times 5^2 + 2 \times 1^2) = 189$

To estimate the second moment of the stream with limited amount of main memory space we can use Alon-Matias-Szegedy algorithm. Here, more the space we use, the more accurate the estimate will be. In this algorithm, we compute the number of variables X . For each variable X , we store when a particular element of the universal set, which we refer to as $X.\text{element}$ and the value of the integer variable $X.\text{value}$. To find the value of a variable X , we select the position in the stream between 1 and n randomly. If element is found in set $X.\text{element}$ then initialize $X.\text{value}$ to 1. Likewise, we read the stream, add 1 to $X.\text{value}$ each time we encounter another occurrence of $X.\text{element}$. Technically, the estimates of the second and higher moments assumes that the stream length n is a constant and it grows with time. Here, we store only the values of variables and multiply some function of that value by n when it is time to estimate the moment.

4.8 Counting ones in a Window

Now, let us see the counting problems for streams. Suppose we have a window of length N on a binary stream and want to find out how many 1's is there in the last k bits? for any $k \leq N$. As we know that practically we cannot afford to store entire window of a stream in memory, so to calculate number on 1's in last k -bits we are going to use approximation algorithm which is explained in a subsequent section.

For a given problem for finding number of 1's in last k -bits, it is necessary to store all N bits of the window along with the representation as fewer than N bits could not work. Since there are 2^N sequences of N bits with fewer than 2^N representations, there must be two different bit strings x and y that have the same representation and if $x \neq y$ then they must differ in at least one bit

4.8.1 The Datar-Gionis-Indyk-Motwani (DGIM) Algorithm

The DGIM algorithm is used to find the number 1's in a data set. This algorithm uses $O(\log_2 N)$ bits to represent a window of N bit, allows to estimate the number of 1's in the window with and error of no more than 50 %. In this, each bit of the stream has a timestamp which signifies the position in which it arrives. The first bit has timestamp 1, the second has timestamp 2, and so on.

As we need to distinguish positions within the window of length N , we shall represent timestamps modulo N which can be represented by $\log_2 N$ bits. To store the total number of bits we ever seen in the stream, we need to determine the window by timestamp modulo N . For that, we need to divide the window into buckets consisting of timestamp of its right (most recent) end and the number of 1's in the bucket. This number must be a power of 2, and we refer to the number of 1's as the size of the bucket. To represent a bucket, we need $\log_2 N$ bits to represent the timestamp which is modulo N of its right end. To represent the number of 1's we only need $\log_2 \log_2 N$ bits. Thus, $O(\log N)$ bits suffice to represent a bucket. There are six rules that must be followed when representing a stream by buckets.

- A) The right side of the bucket should always start with 1 as if it starts with a 0, it is to be neglected. for example, 1001011 here a bucket of size would be 4 as it is having four 1's and starting with 1 on it's right end i.e. Every position with a 1 is in some bucket.
- B) Every bucket should have at least one 1, else no bucket can be formed i.e. Every position with a 1 is in some bucket
- C) No position is in more than one bucket.
- D) There are one or two buckets of any given size, up to some maximum size.

- E) All buckets sizes must be in a power of 2.
- F) Buckets cannot decrease in size as we move to the left.

Suppose, given stream is $\dots 1011011000101110110010110$. The bitstream divided into buckets following the DGIM rules is shown in Fig. 4.8.1.

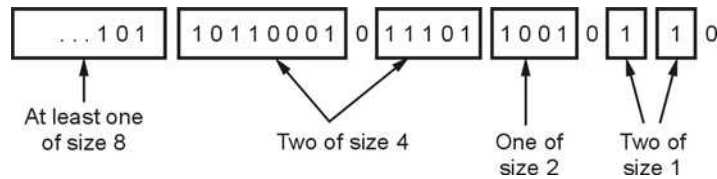


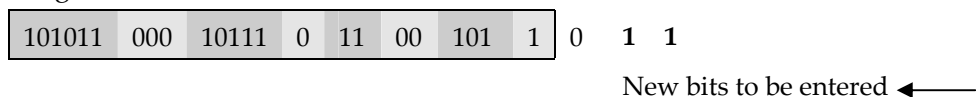
Fig. 4.8.1 : Bitstream divided into buckets following the DGIM rules

For Example : Suppose the input stream bit is $\dots 101011000101110110010110$, so estimate the total number of 1's and number of buckets. Here window size is $N = 24$.

Now, create a bucket whose rightmost bit would be 1. In our example we found 5 buckets as shown below.

101011	000	10111	0	11	00	101	1	0
Bucket size 4 i.e. $2^2=4$		Bucket size 4 i.e. $2^2=4$		Bucket size 2 i.e. $2^1=2$		Bucket size 2 i.e. $2^1=2$	Bucket size 1 i.e. $2^0=1$	

Here, when new bit comes in then drop last bucket if its timestamp is prior to N time before current time. If the new bit arrived is 0 with a time stamp 101, then there are no changes needed in the buckets but if the new bit that arrives is 1, then we need to make some changes.



So current bit is 1 then create a new bucket of size 1 and make the current timestamp and size to 1. If there was only one bucket of size 1, then nothing more needs to be done. However, if there are now three buckets of size 1 (buckets with timestamp 100,102,103) then combine the leftmost(oldest) two buckets of size 2 as shown below.

101011	000	10111	0	11	00	101	1001	1	1
Bucket size 4		Bucket size 4		Bucket size 2		Bucket size 2	Bucket size 2	Bucket size 1	

To combine any two adjacent buckets of the same size, replace them by one bucket of twice the size. The timestamp of the new bucket is the timestamp of the rightmost of the two buckets. By performing combining operation on buckets, the resulting buckets would be

101011	000	10111	1100101	1001	1	1
Bucket size 4		Bucket size 4	Bucket size 4	Bucket size 2	Bucket size 1	

Now, sometimes combining two buckets of size 1 may create a third bucket of size 2. If so, we combine the leftmost two buckets of size 2 into a bucket of size 4. This process may ripple through the bucket sizes. Here, continues the process until current timestamp-leftmost bucket timestamp of window is $< N$ i.e. 24.

So finally, by counting the sizes of the buckets in the last 20 bits, we get solution to the problem i.e. 11 ones.

As each bucket can be represented by $O(\log_N)$ bits. If the window has length N , then there are no more than N 1's. So, if there are $O(\log_N)$ buckets then the total space required for all the buckets representing a window of size N is $O(\log_2 N)$. The solution for the problem to find how many 1's there are in the last k bits of the window, for some $1 \leq k \leq N$. Find the bucket b with the earliest times tamp that includes at least some of the k most recent bits then estimate the number of 1's to be the sum of the sizes of all the buckets to the right (more recent) than bucket b , plus half the size of b itself.

To add new bit in a window of length N represented by buckets, we may need to modify the buckets. So, with satisfying the DGIM conditions, first, whenever a new bit enters check the leftmost bucket. If its timestamp has now reached the current timestamp minus N , then this bucket no longer has any of its 1's in the window. Therefore, drop it from the list of buckets. or create a new bucket with the current timestamp and set its size to 1. However, if there are now more buckets of size 1 then we need to fix this problem by combining the leftmost two buckets of size 1. To combine any two adjacent buckets of the same size, replace them by one bucket of twice the size. Here, the timestamp of the new bucket is the timestamp of the rightmost (later in time) of the two buckets. As a result, any new bit can be processed in $O(\log_N)$ time.

4.9 Decaying Window

The decaying window is used for finding the most common "recent" elements in the streams. Suppose, a stream consist of the elements $a_1, a_2, \dots, a_t$, where a_1 is the first element to arrive and a_t is the current element. Let c be a small constant, such as 10^{-6} to 10^{-9} . Therefore, the exponentially decaying window for this stream would be

$$\sum_{i=0}^{t-1} a_{t-i} (1-c)^i$$

In decaying window, it is easier to adjust the sum exponentially than sliding window of fixed length. The effect of this definition is to spread out the weights of the stream elements as far back in time as the stream goes. In sliding window, the element that falls out of the window each time a new element arrives needs to be taken care. In contrast, a fixed window with the same sum of the weights, $1/c$, would put equal weight 1 on each of the most recent $1/c$ elements to arrive and weight 0 on all previous elements which is illustrated in Fig. 4.9.1. However, when a new element $at+1$ arrives at the stream input, we first multiply the current sum by $1 - c$ and then add $at+1$.

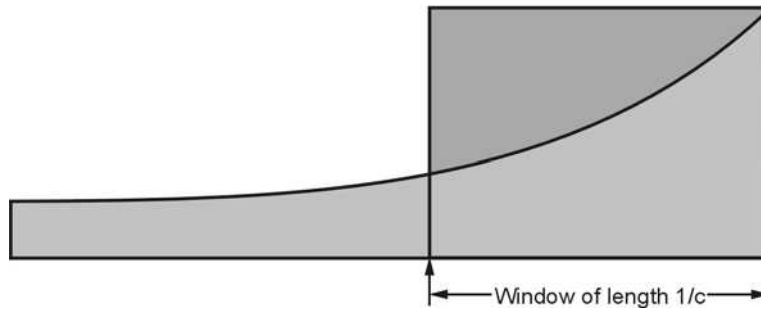


Fig. 4.9.1 : Decaying window

In this method, each of the previous elements get moved one position further from the current element, so its weight is multiplied by $1 - c$. Further, the weight on the current element is $(1 - c)^0 = 1$, so adding $at+1$ is the correct way to include the new element's contribution.

4.10 Real Time Analytics Platform (RTAP)

A real-time analytics platform enables organizations by helping them to extract the valuable information and trends from most out of real-time data. Such platforms help in measuring data from the business point of view in real time. An ideal real-time analytics platform would help in analyzing the data, correlating it and predicting the outcomes on a real-time basis. It helps organizations in tracking the things in real time, thus helping them in the decision-making process as well as connect the data sources for better analytics and visualization. The RTAP is related to responsiveness of data which needs to be processed immediately upon generated, sometimes need to update the information at the same rate at which it gets received. The RTAP analyzes the data, correlates and predicts the outcomes in real-time and helps timely in decision making.

As we know the social medias like Facebook and Twitter generate petabytes of real-time data. This data must be harnessed to provide real-time analytics to make better business decisions. Further in today's context, billions of devices are connected to the internet such as mobile phones, personal computers, laptops, wearable medical devices,

smart meters with huge number of new data sources. The Real-time analytics will leverage information from all these devices to apply analytics algorithms and generate automated actions within milliseconds of a trigger. The Real-Time analytics platform composed of three components namely :

Input : which is generated upon the event happens (like new sale, new customer, someone enters a high security zone etc.)

Processing unit : which capture the data of the event, and analyze the data without leveraging resources that are dedicated to operations. It also involves executing different standing and ad-hoc queries over streamed data and

Output : that consume this data without disturbing operations, explore it for better insights and generates analytical results by means of different visual reports over the dedicated dashboard. The general architecture of Real-Time Analytics Platform is shown in Fig. 4.10.1.

The various requirements for real-time analytics platform are as follows :

1. It must support continuous queries for real-time events.
2. It must consider the features like, robust- ness, fault tolerance, low-latency reads and updates, incremental analytics and learning and scalability.
3. It must have improved the in-memory transaction speed.
4. It should quickly move the not needed data into secondary disk for persistent Storage.
5. It must support distributing data from various sources with speedy processing.

(1) Capturing or collecting the data

(2) Processing real-time data as per flow

(3) Explore and visualize the data

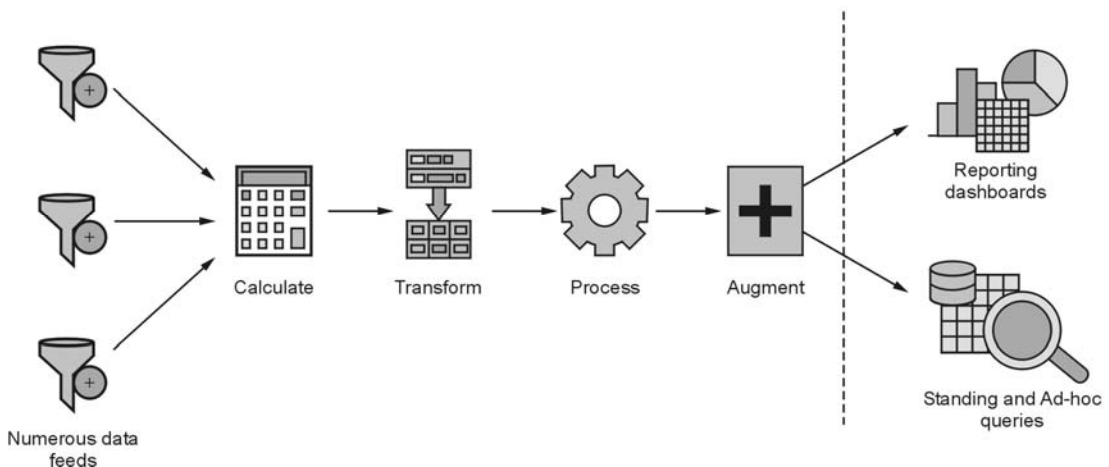


Fig. 4.10.1 : Architecture of Real-Time Analytics Platform

The basic building blocks of Real Time Streaming Platform are shown in Fig. 4.10.2. The streaming data is collected from various flexible data sources by producing connectors which move and receive data from the sources to the queuing system. The queuing system is faulty tolerance and persistent in nature. The streamed data then buffered to be consumed by the stream processing engine. The queuing system is a high-throughput, low latency system which provides high availability and fail-over capabilities. There are many technologies that support real-time analytics, such as :

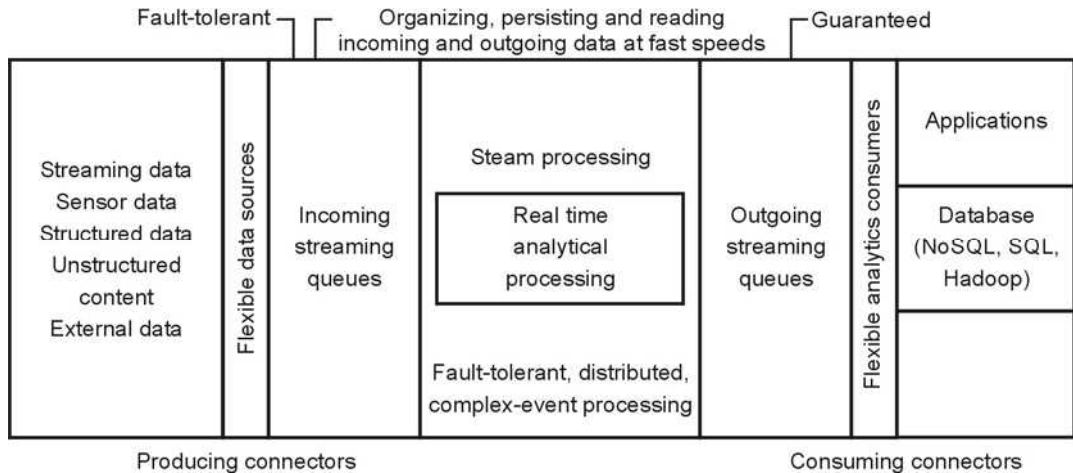


Fig. 4.10.2 : Basic building blocks of Real-Time Analytics Platform

1. **Processing In Memory (PIM)**, a chip architecture in which the processor is integrated into a memory chip to reduce latency.
2. **In-database Analytics**, a technology that allows data processing to be conducted within the database by building analytic logic into the database itself.
3. **Data Warehouse Appliances**, combination of hardware and software products designed specifically for analytical processing. An appliance allows the purchaser to deploy a high-performance data warehouse right out of the box.
4. **In-memory Analytics**, an approach to querying data when it resides in Random Access Memory (RAM), as opposed to querying data that is stored on physical disks.
5. **Massively Parallel Programming (MPP)**, the coordinated processing of a program by multiple processors that work on different parts of the program, with each processor using its own operating system and memory.

Some of the popular Real Time Analytics Platforms are :

- **IBM Info Streams** : It is used as a streaming platform for analyzing broad range of real-time unstructured data like text, videos, geospatial images, sensors data etc.

- SAP HANA : It is a streaming analytical tool that allows SAP users to capture, stream and analyze data with active event monitoring and event driven response to applications.
- Apache Spark : It is a streaming platform for big data analytics in real-time developed by Apache.
- Cisco Connected Streaming Platform : It is used for finding the insights from high velocity streams of live data over the network with multiple sources with enabled immediate actions.
- Oracle Stream Analytics : It provides graphical interface to performing analytics over the real-time streamed data.
- Google Real Time Analytics : It is used for performing real-time analytics over the cloud data collected over different applications.

4.10.1 Applications of Realtime Analytics Platforms

There are many real-time applications which uses realtime analytics platforms, some of them are listed below :

- Click analytics for online product recommendation
- Automated event actions for emergency services like fires, accidents or any disasters in the industry
- Notification for any abnormal measurement in healthcare which requires immediate actions
- Log analysis for understanding user's behavior and usage pattern
- Fraud detection for online transactions
- Push notifications to the customers for location-based advertisements for retail
- Broadcasting news to the users which are relevant to them

4.11 Real Time Sentiment Analysis

The Sentiment Analysis (also referred as opinion mining) is a Natural Language Processing and Information Extraction task that aims to obtain the feelings expressed in positive or negative comments, questions and requests, by analyzing a large number of data over the web. In real-time sentimental analysis, the sentiments are collected and analyzed in real time with live data over the web. It uses natural language processing, text analysis and computational linguistics to identify and extract subjective information in source materials. The goal of sentiments analysis is to allows organizations, political parties and common people to track sentiments by identifying feelings, attitude and state of mind of people towards a product or service and classify them as positive, negative and neutral from the tremendous amount of data in the form of reviews, tweets,

comments and feedback with emotional states such as “angry”, “sad” and “happy”. It tries to identify and extract sentiments within the text. The analysis of sentiments can be either document based where the sentiment in the entire document is summarized as positive, negative or objective or can be sentence based where individual sentences, bearing sentiments, in the text are classified.

Sentiment analysis is widely applied to reviews and social media for a variety of applications, ranging from marketing to customer service. In the context of analytics, sentiment analysis is “the automated mining of attitudes, opinions and emotions from text, speech and database sources”. With the proliferation of reviews, ratings, recommendations and other forms of online expression, online opinion has turned into a kind of virtual currency for businesses looking to market their products, identify new opportunities and manage their reputations.

Some of the popular applications of real-time sentiment analysis are,

- 1) Collecting and analyzing sentiments over the Twitter. As Twitter has become a central site where people express their opinions and views on political parties and candidates. Emerging events or news are often followed almost instantly by a burst in Twitter volume, which if analyzed in real time can help explore how these events affect public opinion. While traditional content analysis takes days or weeks to complete, real time sentiment analysis can look into the entire Twitter traffic about the election, delivering results instantly and continuously. It offers the public, the media, politicians and scholars a new and timely perspective on the dynamics of the electoral process and public opinion.
- 2) Analyzing the sentiments of messages posted to social networks or online forums can generate countless business values for the organizations which aim to extract timely business intelligence about how their products or services are perceived by their customers. As a result, proactive marketing or product design strategy can be developed to effectively increase the customer base.
- 3) Tracking the crowd sentiments during commercial viewing by advertising agencies on TVs and decide which commercials are resulting in positive sentiments and which are not.
- 4) A news media website is interested in getting an edge over its competitors by featuring site content that is immediately relevant to its readers where they use social media to know the topics relevant to their readers by doing real time

sentiment analysis on Twitter data. They Specifically, to identify what topics are trending in real time on Twitter, they need real-time analytics about the tweet volume and sentiment for key topics

- 5) In Marketing, the real-time sentiment analysis can be used to know the public reactions on product or services supplies by an organization. The analysis is performed on which product or services they like or dislike and how they can be improved,
- 6) In Quality Assurance, the real-time sentiment analysis can be used to detect errors in your products based on your actual user's experience.
- 7) In Politics, the real-time sentiment analysis can be used to determine the views of the people regarding specific situations on which they angry or happy.
- 8) In Finances, the real-time sentiment analysis tries to detect the sentiment towards a brand, to anticipate their market moves

The best example of real time sentiment analysis is predicting the pricing or promotions of a product being offered through social media and the web. The solution for price or promotion prediction can be implemented software solutions like Radar (Real-Time Analytics Dashboard Application for Retail) and Apache Storm. The RADAR is the software solution for retailers built using a Natural Language Processing (NLP) based Sentiment Analysis engine that utilizes different Hadoop's technologies including HDFS, Apache Storm, Apache Solr, Oozie and Zookeeper to help enterprises maximize sales through databased continuous re-pricing. Apache Storm is a distributed real-time computation system for processing large volumes of high-velocity data. It is part of the Hadoop ecosystem. Storm is extremely fast, with the ability to process over a million records per second per node on a cluster of modest size. Apache Solr is another tool from the Hadoop ecosystem which provides highly reliable, scalable search facility at real time. RADAR uses Apache STORM for real-time data processing and Apache SOLR for indexing and data analysis. The generalized architecture of RADAR for retail is shown in Fig. 4.11.1.

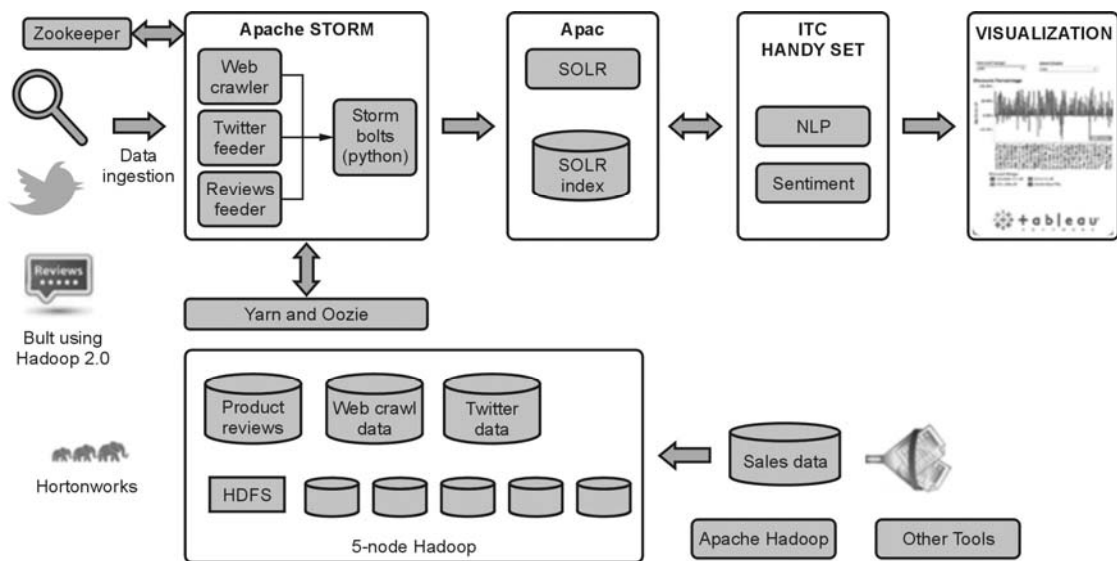


Fig. 4.11.1 : Generalized architecture of RADAR for retail

For retailers, the RADAR can be used to customize their environment so that they can track the following for any number of products / services in their portfolio based on Social Sentiment for each product or service they are offering and competitive pricing/promotions being offered through social media and the web. With this solution, retailers can create continuous re-pricing campaigns and implement them real-time in their pricing systems, track the impact of re-pricing on sales and continuously compare it with social sentiment.

4.12 Stock Market Predictions

Stock market prediction is the act of trying to determine the future value of a company stock or other financial instrument traded on an exchange. The successful prediction of a stock's future price could yield significant profit.

Predicting stock prices is a challenging problem in itself because of the number of variables which are involved. The stock market process is full of uncertainty and it's affected by many factors. Hence the stock market prediction is one of the important exertions in business and finance. As it produces large amount of data every day, it is very difficult for an individual to consider all the current and past information for predicting future trend of a stock.

Traditionally, stock market prediction algorithms used to check historical stock prices and try to predict the future using different models. The traditional approach is not effective in a real time because, as stock market trends continually changes based upon economic forces, regulations, competition, new products, world events and even (positive or negative) tweets are all factors to affect stock prices therefore. Thus, predicting the stock prices using real-time analytics is the necessity. The generalized architecture for real-time stock prediction has three basic steps, as shown in Fig. 4.12.1.

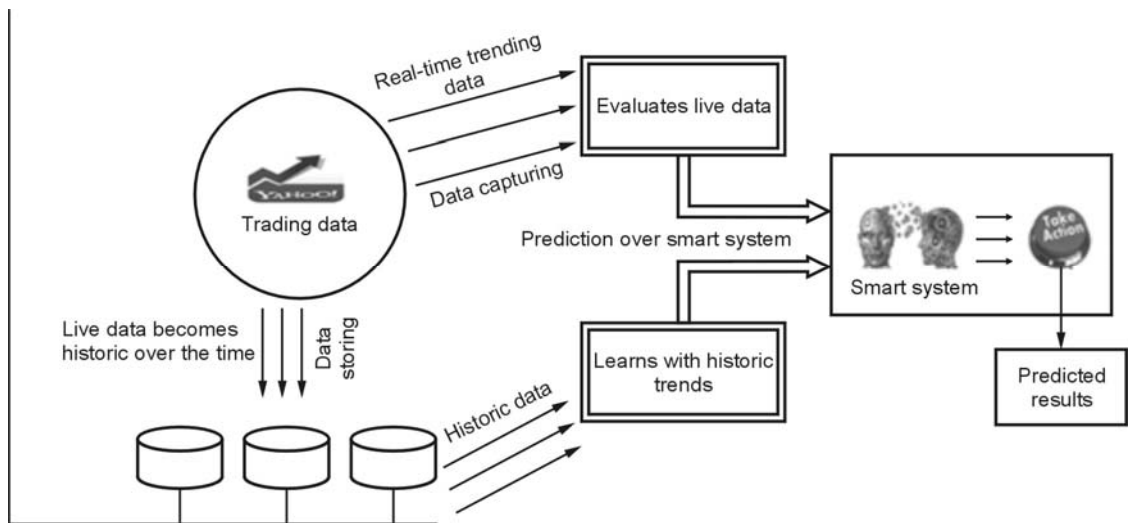


Fig. 4.12.1 : Generalized architecture for real-time stock prediction

There are three basic components :

1. In the first step, the incoming real-time trading data is captured and stored into a persistent storage as it becomes historical data over the period of time.
2. Secondly, the system must be able to learn from historical trends in the data and recognize patterns and probabilities to inform decisions.
3. Third, the system needs to do a real-time comparison of new, incoming trading data with the learned patterns and probabilities based on historical data. Then, it predicts an outcome and determines an action to take.

A more detailed picture with machine learning approach for stock prediction is given in Fig. 4.12.2.

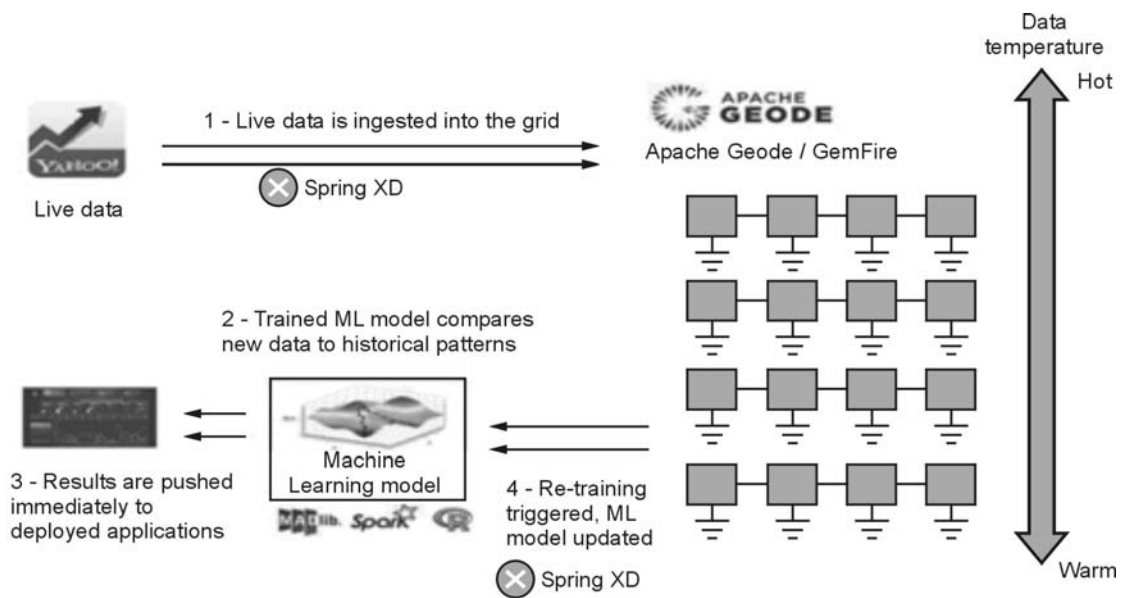


Fig. 4.12.2 : Detailed representation of real-time stock prediction using machine learning

The following steps are followed :

1. The Live data, from Yahoo! Finance or any other finance news RSS feeds is read and processed. The data is then stored in memory with a fast, consistent, resilient, and linearly scalable system.
2. Using the live, hot data from Apache Geode, a Spark MLlib application creates and trains a model, comparing new data to historical patterns. The models could also be supported by other toolsets, such as Apache MADlib or R.
3. Results of the machine learning model are pushed to other interested applications and also updated within Apache Geode for real-time prediction and decisioning.
4. As data ages and starts to become cool, it is moved from Apache Geode to Apache HAWQ and eventually lands in Apache Hadoop™. Apache HAWQ allows for SQL-based analysis on petabyte-scale data sets and allows data scientists to iterate on and improve models.
5. Another process is triggered to periodically retrain and update the machine learning model based on the whole historical data set. This closes the loop and creates ongoing updates and improvements when historical patterns change or as new models emerge.

The most common advantages of stock prediction using big data approach are

- It stabilizes the online trading
- Real-time data analysis with a rapid speed

- Improves the relationship between investors and stock trading firms
- Provides the best estimation of outcomes and returns :
- Mitigate the probable risks on stock trading online and make a right investment decision
- Enhances the machine learning ability to produces accurate predictions

4.13 Graph Analytics

The Big data analytics systems is intended to provide different tools and platforms that can support various analytic techniques and can be adapted to overcome the challenges in existing system. The graph analytics is one of the techniques in which both structured and unstructured data is supports from various sources to enable analysts to probe the data in an undirected manner. It is adopted by many organizations because of simpler visualization of data over the past data warehouse and analytics techniques.

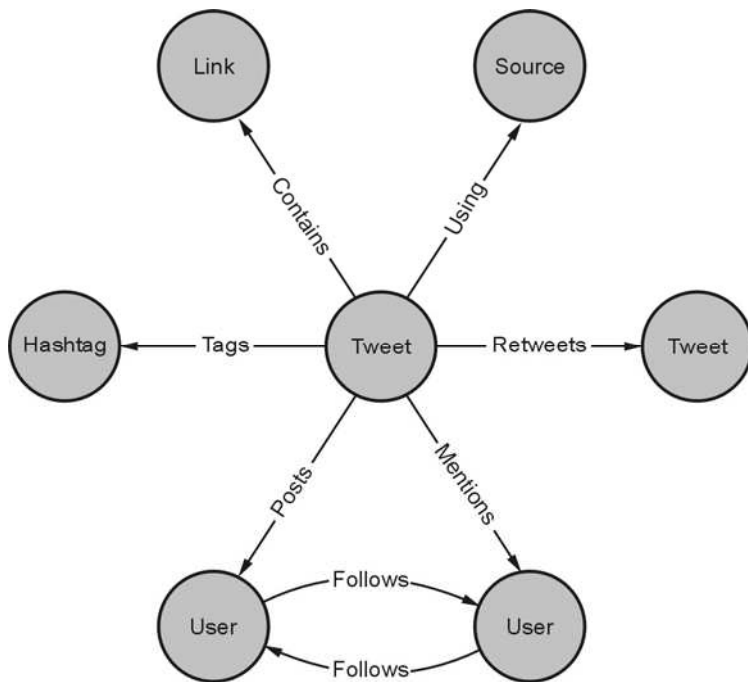


Fig. 4.13.1 : Graph representation

It is composed of numerous individual entities and different relationships that connect those entities. It consists of collection of vertices referred as nodes to represent entities, connected by edges referred as links or connections to represent relationships between entities. The Fig. 4.13.1 shows a typical graph representation, in which the edges between vertices represents the nature of the relationship with direction to the entities. In typical

graph analysis, the labeled vertices indicate the types of entities that are related while labeled edges are used to represent the nature of the relationship, while multiple relationships between pairs of vertices are represented by multiple edges between pair of vertices.

In graph analytics, the directed graph can be represented by triplet consist of subject which is the source point of the relationship, an object is the target point of relationship, and a predicate that represents type of the relationship. Therefore, the database which support these triplets is called a semantic database. The graph model supports all the types of entities and their relationship. The graph model composed of different models namely communication models that represents communication across a community triggered by a specific event, influence model that represents entities holding influential sites within a network for intermittent periods of time, Distance modeling for analyzing the distances between sets of entities like finding the strong correlations between occurrences of sets of statistically improbable phrases and collaborative model that uses isolated groups of individuals that share similar interests. The graph analytics is mainly used for business problems that has characteristics like Adhoc nature of the analysis, absence of structure in the problem, embedded knowledge in the network, connectivity problem, predictable performance, undirected discovery and flexible semantics.

4.13.1 Features of a Graph Analytics

The Graph analytics encompass following features

- a) **Easier visualization** : The graph analytics has visualization tools that represents easier discovery of valuable information and highlight their value.
- b) **Seamless data intake** : It provides a seamless capability to easily collect and use data from a variety of different sources.
- c) **Simple data integration** : A semantics-based approach in graph analytics allows to easily integrate different sets of data that do not have predetermined structure.
- d) **Seamless workflow integration** : A graph analytics platform provides seamless approaches for workflow integration which are segregated from the existing reporting and analytics environments having limited value when incorporating the results from different environments.
- e) **Multithreading** : The graph analytics platform has fine-grained multithreading approaches which allows exploration of different paths for creating, managing and allocating threads to available nodes on a parallel processing architecture.

- f) **Standardize representation** : The graph analytics platforms has built-in resource description framework standard like RDF and ontology to use triplets for representing the graph.
- g) **Built-in inferencing mechanisms** : The graph analytics platform has methods for finding the insights derived from the embedded relationships by using different built-in inference mechanisms for the deduction of new information.

The graph analytics applications run different algorithms to traverse or analyze the graphs for finding interesting patterns within them. Those patterns are useful for getting new business opportunities, increasing revenue, detecting frauds and identifying different security risks.

The different approaches used by graph analytics algorithm are given as below

- i) **Path analysis** : This approach examines the shapes and distances of the diverse paths that connect entities within the graph.
- ii) **Clustering analytics** : This approach examines the properties of the vertices and edges to recognize features of entities that can be used to group them together.
- iii) **Pattern detection and analysis** : This approach provide methods for finding the inconsistent or unexpected patterns within a graph for analysis.
- iv) **Probabilistic analysis** : This approach provides different graphical models for probabilistic analysis on various applications like risk analysis, speech recognition, medical diagnosis, protein structure prediction etc. using Bayesian networks.
- v) **Community analysis** : In this approach the graph structures are traversed in search of groups of entities connected in close ways.

The graph analytics is used in many applications like health care, where patient's health records like collections of medical histories, prescription records, laboratory results and clinical records from many different sources are analyzed and based on that the rapid assessment of therapies are provided for other patients who are facing the same medical problem. The cyber security is another application where the patterns of attack are recorded and actions are taken against those attacks. The third application is concept-based correlations which is used for finding contextual relationships between different entities like finding the fraud analysts by evaluating financial irregularities across multiple-related organizations.

Unlike many advantages, there are some limitations of graph analytics like complexity of graph partitioning, unpredictability of graph memory accesses, dynamic interactions with graphs, and unpredicted growth of a graph models.